

Airport

Bring Arrow Flight services into your SQL.



Install & load

```
INSTALL airport FROM community;
LOAD airport;
```

01 At a glance

Attach an Arrow Flight service as a DuckDB catalog, then query its tables alongside local data. Discover available flights, fetch a descriptor directly, and invoke server-defined actions when a full catalog is unnecessary.

Flight server → Arrow stream → DuckDB catalog

02 Catalogs, flights & actions

ATTACH ... (TYPE airport)

Mount a compatible Flight catalog under a DuckDB alias.

airport_databases(url)

Discover databases exposed by the server.

airport_flights(url)

List available flight descriptors and metadata.

airport_take_flight(url, descriptor)

Read a specific flight directly as a table.

airport_action(url, action, body)

Invoke a server-defined action; available actions vary.

CREATE SECRET ... (TYPE airport)

Scope a bearer token to a Flight URL prefix.

airport_version()

Identify the loaded Airport build for diagnostics.

03 Keep in mind

Capabilities come from the server

Catalog discovery, filter pushdown and writes depend on the Flight server's Airport-compatible interfaces. A service using Arrow internally does not necessarily expose those interfaces. Check its schema and capabilities before attaching.

04 Discover before attaching

REQUIRES SETUP

```
SELECT * FROM airport_databases(
  'grpc://localhost:8815'
);
SELECT * FROM airport_flights(
  'grpc://localhost:8815'
);
```

Requires a local compatible Flight server listening on port 8815.

05 Query an attached catalog

REQUIRES SETUP

```
ATTACH 'grpc://localhost:8815' AS remote
(TYPE airport);
SHOW TABLES FROM remote.public;
SELECT user_id, count(*) AS n
FROM remote.public.activity
GROUP BY user_id
ORDER BY n DESC
LIMIT 10;
DETACH remote;
```

Assumes the server exposes public.activity with a user_id column.

06 Identify the loaded build

```
SELECT airport_version(), airport_user_agent();
```

Local diagnostics; no server connection required.

07 In practice

Authenticate by URL scope

Use an airport secret with scope and auth_token for protected services. Use grpc+tls for TLS endpoints. Server-defined descriptors, table names and actions must match the service you connect to.

[Full docs & examples ↗](#) [Source ↗](#)